

Interpreting the results of the Naive Bayes model

- **Introduction and objectives :**

The Naive Bayes model is a simple powerful probabilistic classifier based on Bayes' Theorem, widely used in machine learning for tasks like spam detection, sentiment analysis, and document classification. It assumes that the features in the data are conditionally independent, making it computationally efficient and effective, especially for large datasets. **This project aims to implement and understand the Naive Bayes algorithm**, exploring its strengths and limitations in various classification tasks.

During our work on this model, **two types of Naive Bayes, namely Gaussian and Bernoulli**, were applied to see how different these two types were, and whether their results after building the model would be completely different or not.

Our Objectives in this Task is :

1. Implement the Naive Bayes classifier using Python.
2. Understand and apply Bayes' Theorem in a machine learning context.
3. Test the model on a dataset to evaluate its accuracy and performance.
4. Compare the results with other machine learning models.
5. Identify the advantages and limitations of using Naive Bayes in classification tasks.

GAUSSIAN NAIVE BAYES CLASSIFIER

"Gaussian" because this is a normal distribution

This is our prior belief

$$P(\text{class} | \text{data}) = \frac{P(\text{data} | \text{class}) \times P(\text{class})}{P(\text{data})}$$

We don't calculate this in naive bayes classifiers

Implementing Naive Bayes (Gaussian & Bernoulli) on the Titanic Dataset :

1. **Data Preprocessing** : In the beginning, before starting to build the model, we processed our dataset, and we found that it contains many missing and annoying values, it was handled with high efficiency, due to the importance of cleaning the data and making it ready to build the model, and this will make the data of high quality, so our data in this case is ready to start the task.
- 2- **Split The dataset** : We divided the dataset into two subsets: training and test groups. The `train_test_split` function is used to divide data randomly, where 70% of the data (`X_train` and `y_train`) is allocated to model training, and the remaining 30% (`X_test` and `y_test`) are reserved to evaluate model performance. The parameter `test_size=0.3`

specifies the percentage of the dataset to include in the test split. **Setting random_state = 42** ensures that split is consistent and repeatable across different operations, providing a stable basis for comparison and evaluation.

3- **Build the Model** : Build the model using GaussianNB() Algorithms , after splitting the data , use the fit method to train the parameter x and y .

4- **Evaluation the Model** : This includes:

accuracy_score, classification_report, confusion_matrix ,
To calculate the accuracy rate of the model and show the expected results and the success or failure rate

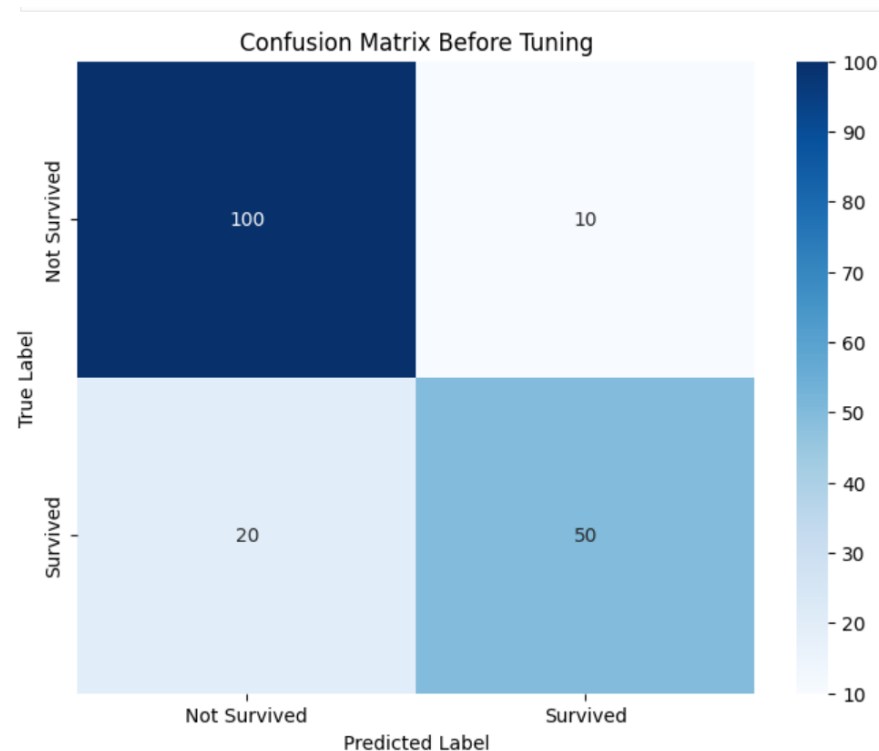
The result of Gaussian types :

```
Accuracy: 0.79
Classification Report:
              precision    recall  f1-score   support

     0           0.83       0.82       0.82       157
     1           0.75       0.76       0.75       111

 accuracy                   0.79       268
 macro avg           0.79       0.79       0.79       268
 weighted avg        0.80       0.79       0.79       268
```

Confusion Matrix before optimizing the model



Confusion Matrix Explanation :

True Negatives (TN): 100

Predicted "Not Survived" and actually "Not Survived."

False Positives (FP): 10

Predicted "Survived" but actually "Not Survived."

False Negatives (FN): 20

Predicted "Not Survived" but actually "Survived."

True Positives (TP): 50

Predicted "Survived" and actually "Survived."

Not Survived: The model correctly identified 100 cases and incorrectly identified 10 cases.

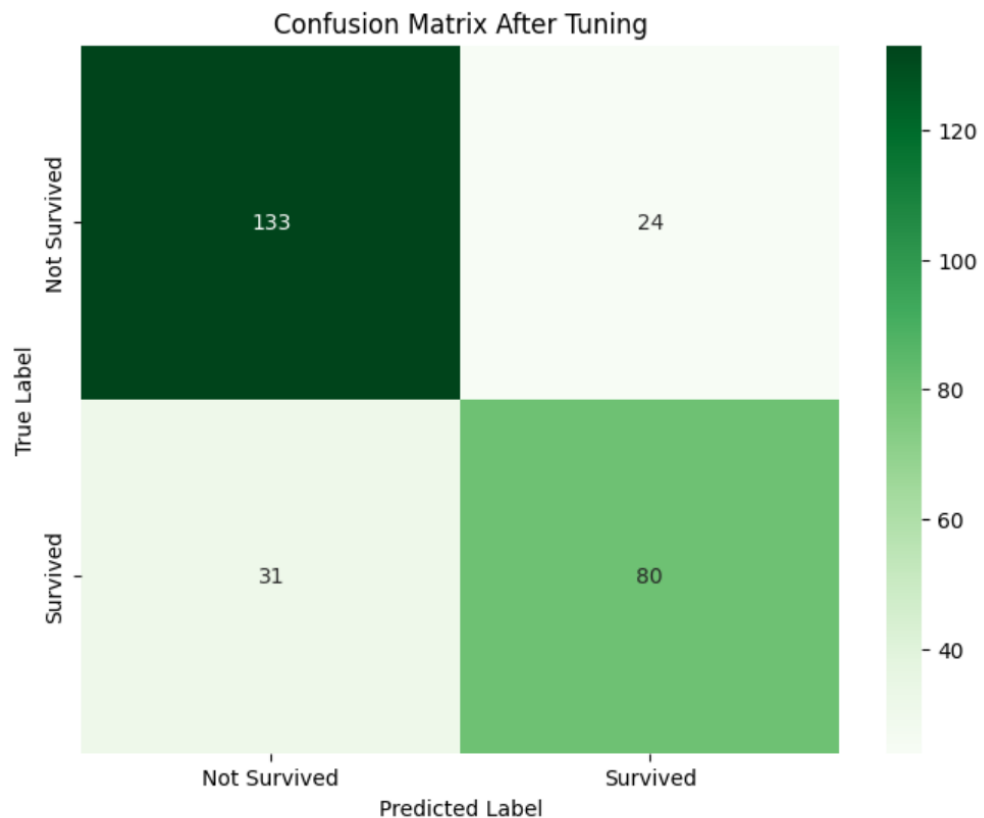
Survived: The model correctly identified 50 cases but missed 20 cases.

Performance Insights :

High true negatives (100) and true positives (50) indicate good performance.

There are some misclassifications (10 false positives and 20 false negatives), which could be improved with tuning.

Confusion Matrix After optimizing the model



Explanation the result :

This confusion matrix shows the performance of a classification model after tuning. Here's a simple breakdown:

- **True Negatives (TN):** 133
 - Predicted "Not Survived" and actually "Not Survived."
- **False Positives (FP):** 24
 - Predicted "Survived" but actually "Not Survived."
- **False Negatives (FN):** 31
 - Predicted "Not Survived" but actually "Survived."
- **True Positives (TP):** 80
 - Predicted "Survived" and actually "Survived."

Summary

- **Not Survived:** The model correctly identified 133 cases and incorrectly identified 24 cases.
- **Survived:** The model correctly identified 80 cases but missed 31 cases.

Performance Insights

- There's an improvement in true negatives (133) and true positives (80) compared to the previous matrix.

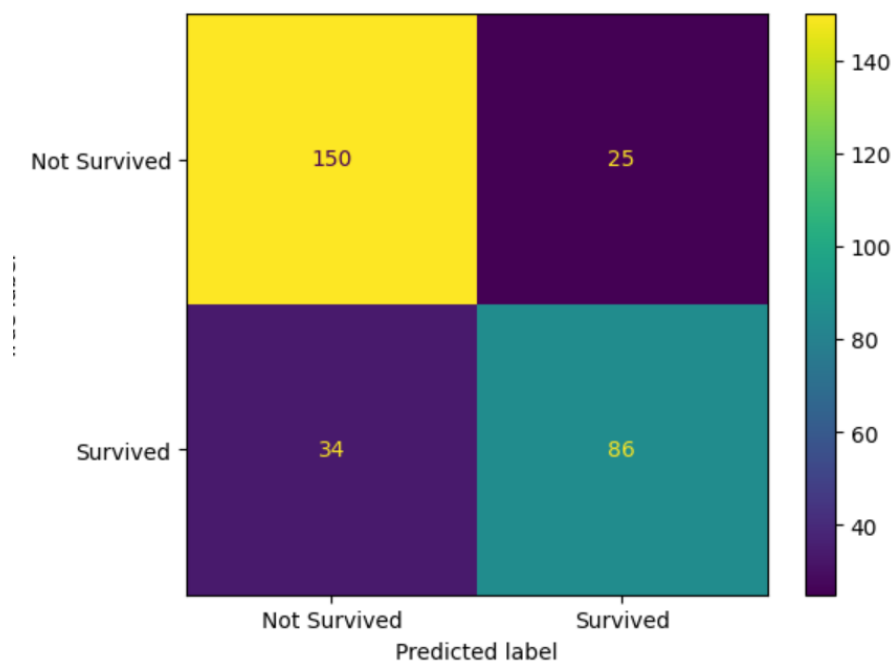
- The number of false positives (24) and false negatives (31) is higher than in the earlier model, indicating some misclassifications, but overall performance has improved.

The result of Bernoulli types :

Bernoulli Naive Bayes is a subcategory of the Naive Bayes Algorithm. It is used for the classification of binary features such as 'Yes' or 'No', '1' or '0', 'True' or 'False' etc. Here it is to be noted that the features are independent of one another. Bernoulli Naive Bayes is basically used for spam detection, text classification, Sentiment Analysis, used to determine whether a certain word is present in a document or not.

Accuracy: 0.80				
Classification Report:				
	precision	recall	f1-score	support
0	0.82	0.86	0.84	175
1	0.77	0.72	0.74	120
accuracy			0.80	295
macro avg	0.79	0.79	0.79	295
weighted avg	0.80	0.80	0.80	295

Confusion Matrix of Bernoulli Classifier



Confusion Matrix Breakdown :

True Negatives (TN): 150

Predicted "Not Survived" and actually "Not Survived."

False Positives (FP): 25

Predicted "Survived" but actually "Not Survived."

False Negatives (FN): 34

Predicted "Not Survived" but actually "Survived."

True Positives (TP): 86

Predicted "Survived" and actually "Survived."

Performance Insights

- The model shows strong accuracy in predicting "Not Survived" with 150 true negatives.

- There are some misclassifications: 25 false positives and 34 false negatives, but overall, the model performs well with a good balance between sensitivity and specificity.

Class – Lab : 1

Group Number : 8

Group members : Yazan Ibrahim Al-Ghamdi – Abdulmalik
Hassan Bajendoh

Thank You !!